A Multi-Harness Corpus of 75,933 Coding-Agent Actions with Per-Action Outcome Labels

Giovanni Pezzin

Abstract

Public datasets for coding agents overwhelmingly score task-level success: whether a patch passes a test suite, whether a repository is modified correctly. They rarely record what happened at the level of the individual tool call. We release a corpus of 75,933 coding-agent actions from 402 sessions across four harnesses (Claude Code, Codex, Cline, OpenCode) on three machines, where every action carries a normalised action kind, an outcome label, and a redacted target identifier. We document the corpus's construction, its redaction procedure, and the label-heterogeneity problem that any consumer must handle: the four harnesses record failure four different ways. We then run the test that would most cheaply undercut the release, and find the opposite: the corpus's fine-grained action-kind taxonomy is load-bearing. Under leave-one-session-out evaluation the 871-way action-kind label yields AUC 0.627–0.751 on three of four harnesses, while the coarser 133-way tool label and a 131-way command prefix label collapse to 0.446–0.525, near chance. A dataset that shipped tool-level granularity would carry almost none of the signal. We release the corpus, the extraction scripts, and the analysis code.

1 Motivation

Benchmarks for coding agents measure outcomes: SWE-bench scores whether a generated patch resolves an issue [Jimenez et al.(2024)], and similar suites score task completion. That framing answers whether an agent succeeded but not *where* it failed. A session of several hundred tool calls that ends in a passing test can still contain dozens of failed commands; a session that fails may have failed at exactly one step.

Per-action outcome data supports a different class of question: predicting which actions will fail from the agent's own history, and comparing failure structure across agents. This is the analysis the corpus is built for, and it is why the outcome label is per-action rather than per-task.

2 Corpus construction

Actions are extracted from harness-native session logs on each machine, and only derived action records leave the machine of origin: no transcripts, no raw command text, no tool output is copied. The four extractors are:

- **Claude Code**: `~/.claude/projects/*.jsonl`, pairing `tool_use` with `tool_result`.
- **Codex**: `~/.codex/sessions/**/rollout-*.jsonl`, carrying a real `exit_code`.
- **Cline**: `~/.cline/data/sessions/*/*.messages.json`, shell results arriving in batches across `result`/`output`/`error`.
- **OpenCode**: `opencode.db` (SQLite), the `part` table, with `state.status`.

Each action record carries: the normalised action `kind`, the `tool`, a salted digest of the target, the output size in bytes, and two outcome flags (`harness_error`, `marker_error`). Session boundaries, the machine, and the harness are preserved.

Two merge defects are documented because they shaped the released data. First, reading only Cline's `text` key yielded a failure rate of 0.005 over 8,301 actions; reading `result`, `output` and `error` as well yielded 0.070, and treating `run_commands` as a single kind collapsed 6,602 actions into one label. The released data uses the corrected extraction, which yields 112 kinds where the broken extractor yielded one. Second, the merge initially labelled harness from the wrong field, tagging every Cline and OpenCode session as Claude Code. Sessions appearing byte-identically on multiple machines are deduplicated by session id: 188 duplicate session records were dropped from 590 raw records.

3 Corpus statistics

Harness	Sessions	Actions	Machine(s)	Failure rate
OpenCode	288	48,460	pansa, omo, feather	0.048
Claude Code	42	14,457	pansa, omo, feather	0.061
Cline	69	12,354	pansa, omo, feather	0.074
Codex	3	662	pansa, omo	0.038
Total	**402**	**75,933**	3 machines	**0.055**

Table 1: Corpus composition. Failure rate is the per-action `marker_error` rate.

The corpus contains 871 distinct action kinds and 133 distinct tools. Sessions are uneven in size: the median session carries 101 actions, and the largest carries 2,014. This clustering matters for any consumer computing uncertainty; see Section 8.

4 Failure is structured by action kind

Failure rate is not uniform across kinds. Among kinds with at least 20 occurrences, the extremes span more than an order of magnitude:

Low-rate kind	n	rate	High-rate kind	n	rate
`bash:grep`	3,024	0.018	`bash:physis_loop`	20	0.450
`bash:praxis_webapp`	2,414	0.020	`bash:examples`	23	0.435
`bash:ls`	2,495	0.027	`bash:apksigner`	54	0.407
`read`	10,082	0.043	`bash:cargo`	1,418	0.333

Table 2: Extremes of per-kind failure rate, kinds with $n \geq 20$.

The pattern is interpretable: read-only and search actions rarely fail, while build, packaging, and process-control actions fail often. This structure is what per-action outcome labels make visible, and it is the corpus's justification.

5 The kill test: does label granularity matter?

A dataset release should be tested against the cheapest objection to its value. Here that objection is: the fine-grained taxonomy is decorative, and a coarse label would have served as well.

We test it directly. Using leave-one-session-out evaluation within each harness, we predict each action's outcome from its label's prior failure rate, computed on all other sessions of that harness. We repeat this at three granularities the dataset could have shipped instead:

- `kind` (871 values) — what we release,
- `tool` (133 values) — the coarser unit,
- `kind_prefix` (131 values) — the command word before the colon, e.g. `bash:git` → `bash`.

Harness	kind (871)	tool (133)	prefix (131)
Claude Code	**0.627**	0.525	0.525
Cline	**0.751**	0.446	0.446
OpenCode	**0.680**	0.491	0.491
Codex	0.461	0.446	0.508

Table 3: Leave-one-session-out AUC by label granularity. Coarse labels do not merely lose a little: they collapse to chance.

The coarse labels collapse to chance on every harness with enough data to measure. This is not a small margin in favour of the fine taxonomy — it is the difference between a signal and no signal. Note also that `tool` is nearly a no-op: 133 tools cannot separate the 871 kinds, and the AUC falls to 0.446–0.525. On Codex, where only three sessions exist, the ordering is reversed and all three values sit near chance; we report that rather than hide it, and it is the reason we recommend a minimum session count for any consumer of the corpus.

6 Label heterogeneity

The four harnesses do not measure the same thing, and consumers must not pool them without deciding what to do about it:

- **Codex** records a real process `exit_code`.
- **OpenCode** records `state.status`, a harness judgement.
- **Claude Code** records a harness-reported `is_error`.
- **Cline** records mostly output markers.

The released corpus keeps both flags. `harness_error` is the harness's own judgement and is available for Codex, OpenCode and Claude Code (corpus rate 0.020). `marker_error` additionally fires on failure markers at line start — Traceback, `error[E0433]`, FAILED, `fatal:`, `panicked at`, `command not found` — and is available for all four (corpus rate 0.055). Consumers wanting the strictest definition should use `harness_error` and accept losing Cline; consumers wanting full harness coverage should use `marker_error` and accept a looser label.

7 Redaction and ethics

Targets are replaced by salted digests, and raw command text is not released. This is not a formality: a scan of the merged corpus found an API key and a bearer token inside recorded commands. Any release of agent-session data must assume that credentials appear in tool arguments, because agents routinely pass them on the command line.

The redaction procedure is:

1. Extract on the machine of origin; only derived records move.
2. Replace every target with a salted digest (`target_failed` still means "this exact target failed before", so the digest is stable within the corpus but not reversible).
3. Store no command text, no tool output, and no file contents.
4. Scan the merged corpus for credential-shaped strings before release.

Residual risk is acknowledged rather than eliminated: output *lengths* are preserved, action kinds include command names, and a determined analyst could in principle infer project names from `bash:` kinds such as `bash:praxis_webapp`. We consider this an acceptable trade for the corpus's usability, and we flag it so consumers can decide otherwise.

8 The inferential unit

75,933 actions is not 75,933 independent observations. Actions cluster within sessions, and sessions cluster within projects. A bootstrap that resamples actions treats a session's 200 actions as 200 independent draws and produces intervals that are too narrow. On the twelve cross-harness transfer comparisons this changes the significance verdict for *four* of them relative to session-level resampling: three pairs become distinguishable from chance that were not, and one that was distinguishable stops being so. It is not a one-way optimism — the same narrowing also hides a below-chance result — but in every case the action-level interval is the narrower one, and it is never the one to report. Any uncertainty estimate on this corpus should resample *sessions*, not actions. We state this as a condition of use because it is easy to get wrong and the corpus's size invites it. The per-pair intervals under both units are stored in `research/bootstrap_unit_check.json`, which is the artifact these numbers come from, and the same file is the canonical source for the counts reported in the failure-priors paper.

9 Limitations

- **One user.** The four harnesses are driven by one person. The "cross-agent" axis is four agents but one human, so the corpus cannot separate agent-specific from user-specific failure structure.
- **One fortnight.** The collection window is short, so seasonal or project-phase effects are uncontrolled.
- **Three machines.** `air`, `pi` and `k1` were unreachable at extraction time and contribute nothing.

- **Label drift.** Section 6's four label definitions introduce an uncontrolled confound into every cross-harness number.
- **Uneven harness coverage.** Codex contributes 3 sessions and 662 actions; conclusions involving Codex are correspondingly weak.

10 Availability and reproduction

- Corpus: `benchmarks/behaviour/corpus-redacted.json.gz` (796 KB, 402 sessions, 75,933 actions).
- Extractors: `benchmarks/behaviour/extract_*.py`.
- Granularity analysis: `research/papers/dataset_baselines.py`, writing `research/dataset_baselines.js`
- Cross-harness transfer analysis built on this corpus: `research/papers/e74_replication.py`.
- The extraction-defect figures in Section 2 (the 0.005-then-0.070 Cline failure rate over 8,301 actions) are quoted from the extraction report `research/E74_FOUR_HARNESSES.md` in the retired archive `.physis-pro-retired-20260920-014539`, which is preserved rather than deleted. They are documentary, not regenerable from the released corpus, since the corpus reflects the corrected extraction.

11 Conclusion

We release a per-action coding-agent corpus spanning four harnesses and three machines, with documented extraction, deduplication and redaction. The release is justified by a test we ran against our own interest: the 871-way action-kind taxonomy carries the signal (AUC 0.627–0.751), and coarsening it to tool or prefix collapses performance to chance (0.446–0.525). The corpus's value is therefore in its granularity, not merely in its size, and we report the assumptions a consumer must accept — one user, one fortnight, four different failure labels, and session-level clustering — rather than leaving them to be discovered.

References

[Jimenez et al.(2024)] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *ICLR 2024*. arXiv:2310.06770.